



CLICKGEO

CURSOS DE GEOTECNOLOGIAS

Introdução ao R Part 1

R é uma linguagem de programação e ambiente de software para computação estatística. É amplamente usado por estatísticos, cientistas de dados e pesquisadores, sendo amplamente conhecido por sua extensibilidade e comunidade ativa.

História da Linguagem R

O R é uma linguagem de programação de alto nível, interpretada e de uso geral. Sua comunidade oferece uma ampla gama de pacotes e ferramentas específicas para diversas aplicações, como ggplot2, dplyr, tidyverse e outros para aplicações em ciência de dados.

O R foi criado por **Ross Ihaka** e **Robert Gentleman** em 1993. Ao longo dos anos, evoluiu com versões que expandiram suas capacidades, incluindo melhorias em manipulação de dados e capacidades gráficas.

Curiosidade:

O nome da linguagem foi inspirada na primeira letra dos nomes de seus criadores, **Ross e Robert**.

Sintaxe Básica

Para aprender uma linguagem de programação, é essencial compreender sua sintaxe básica. No caso do R, isso envolve familiarizar-se com algumas regras e convenções fundamentais que o diferenciam de outras linguagens:

Indentação e Comentários

- **Delimitação de Blocos de Códigos:** Ao contrário do Python, R não usa indentação para delimitar blocos de código. Em R, blocos são delimitados por chaves `{}` ou são definidos dentro de funções.
- **Comentários:** Os comentários no R são indicados pelo símbolo `#`. Tudo que segue o `#` na mesma linha é tratado como um comentário e não é executado.

```
## Comentário
#{
#   Bloco1
#   Bloco2 # Comentário
#   # Comentário
#   Bloco3
#}
```

Variáveis

Em R, as variáveis também são criadas sem a necessidade de declarar explicitamente o tipo de dado, sendo então uma linguagem de tipagem dinâmica.

```
variavel <- 1  
print(class(variavel))
```

```
## [1] "numeric"
```

```
variavel <- "Hello"  
print(class(variavel))
```

```
## [1] "character"
```

```
var_5 <- 5.3  
print(class(var_5))
```

```
## [1] "numeric"
```

Operadores Aritméticos

Os operadores aritméticos no R são usados para realizar cálculos aritméticos e tem o mesmo padrão de qualquer outra linguagem de programação.

- **Adição (+):** Usado para somar dois números ou concatenar strings e outras sequências.
- **Subtração (-):** Empregado para subtrair um número de outro.
- ****Multiplicação (*):** Utilizado para multiplicar dois números.
- **Divisão (/):** Aplicado para dividir um número por outro, resultando em um valor de ponto flutuante.
- **Exponenciação (^) ou (**):** Usado para elevar um número a potência de outro.
- **Divisão Inteira (%/):** Realiza a divisão entre dois números, mas arredonda o resultado para o número inteiro mais próximo inferior.

```
# Adição  
2 + 3
```

```
## [1] 5
```

```
# Subtração  
44 - 3
```

```
## [1] 41
```

```
# Multiplicação  
44 * 12
```

```
## [1] 528
```

```
# Divisão  
70 / 3
```

```
## [1] 23.33333
```

```
# Divisão inteira  
70 %% 3
```

```
## [1] 23
```

```
# Exponenciação  
70 ** 3
```

```
## [1] 343000
```

```
# Exponenciação  
70 ^ 3
```

```
## [1] 343000
```

A ordem das operações em R segue a convenção matemática padrão.

```
# Ordem das operações  
1 + 3 * 2 + (5 ^ (3 - 2))
```

```
## [1] 12
```

Estrutura de Dados

As estruturas de dados são fundamentais na programação em R, pois definem a maneira como os dados são organizados, armazenados e manipulados.

Vetores e Matrizes Para o R, as estruturas de dados mais básicas são os vetores e matrizes. Vetores em R são semelhantes aos arrays em Python, mas são homogêneos, ou seja, todos os elementos devem ser do mesmo tipo, usamos `c()` para criar um vetor.

```
# Criando um vetor em R  
vetor <- c(9, 14, 22, 2)  
print(class(vetor))
```

```
## [1] "numeric"
```

Matrizes em R são bidimensionais e também homogêneas. São semelhantes a listas de listas em Python, mas com a restrição de que todas as sublistas (linhas ou colunas) devem ter o mesmo tamanho e usamos `matrix(minimo:maximo, nrow = linhas)`

```
# Criando uma matriz em R
matriz <- matrix(1:9, nrow = 3)
print(matriz)
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

Booleanos ou Operadores Lógicos Os valores booleanos e operadores lógicos em R funcionam de maneira similar a outras linguagens, representando estados de verdadeiro (TRUE) e falso (FALSE).

```
# Operadores lógicos em R
print(class(TRUE))
```

```
## [1] "logical"
```

```
print(class(FALSE))
```

```
## [1] "logical"
```

```
# Exemplos de operadores de comparação
valor1 <- 5
valor2 <- 10
```

```
# Igualdade
valor1 == valor2
```

```
## [1] FALSE
```

```
# Maior ou igual
valor1 >= valor2
```

```
## [1] FALSE
```

```
# Menor
valor1 < valor2
```

```
## [1] TRUE
```

```
# Diferença
valor1 != valor2
```

```
## [1] TRUE
```

Números Em R, os números são tratados como vetores. Os tipos básicos são números inteiros e float, entretanto o R também suporta os números complexos de forma nativa.

```
# Exemplo de números em R
int_num <- 32L # Inteiro
print(class(int_num))
```

```
## [1] "integer"
```

```
float_num <- 32.0 # Ponto flutuante
print(class(float_num))
```

```
## [1] "numeric"
```

```
complex_num <- 1+4i # Complexo
print(class(complex_num))
```

```
## [1] "complex"
```

Listas As listas podem conter elementos de diferentes tipos. São estruturas de dados muito versáteis em R.

```
# Criando uma lista em R
lista <- list(1, "R", TRUE, 1+4i)
print(lista)
```

```
## [[1]]
## [1] 1
##
## [[2]]
## [1] "R"
##
## [[3]]
## [1] TRUE
##
## [[4]]
## [1] 1+4i
```

```
# Acessando elementos da lista
print(lista[[1]])
```

```
## [1] 1
```

```
print(lista[[2]])
```

```
## [1] "R"
```

Strings As Strings são tratadas como vetores de caracteres eo R possui diversas funções para manipular strings.

```
# Exemplo de string em R
nome <- "João"
print(nome)
```

```
## [1] "João"
```

```
# Concatenação de strings
nome_completo <- paste("João", "Guilherme")
print(nome_completo)
```

```
## [1] "João Guilherme"
```

Fatores Fatores são uma estrutura de dados única em R, usada para armazenar dados categóricos. São semelhantes aos *enums* em outras linguagens como Java.

```
# Criando um fator em R
genero <- factor(c("Masculino", "Feminino", "Masculino", 2))
print(genero)
```

```
## [1] Masculino Feminino Masculino 2
## Levels: 2 Feminino Masculino
```

Data Frames Data frames são talvez a estrutura de dados mais importante em R para análise de dados. São similares a tabelas em bancos de dados ou a DataFrame.

```
# Criando um data frame em R
dados <- data.frame(
  Nome = c("Ana", "João"),
  Idade = c(23, 25),
  Sexo = c("Feminino", "Masculino")
)
print(dados)
```

```
##   Nome Idade   Sexo
## 1  Ana    23 Feminino
## 2 João    25 Masculino
```

Condicionais

As estruturas condicionais `if`, `else` e `else if` em R são partes cruciais da lógica de programação, permitindo o controle do fluxo de execução do programa com base em diferentes condições.

- **if**: Avalia uma condição e, se essa condição for verdadeira (`TRUE`), executa o bloco de código abaixo dela.
- **else**: Captura todas as condições que não foram capturadas pelo bloco `if`. O bloco de código sob `* else` é executado se a condição no `if` for falsa (`FALSE`).
- **else if**: Usado para testar condições adicionais se a condição no `if` não for verdadeira. Pode haver vários blocos `else if` após um `if`

Vamos criar um vetor em R que contém uma variedade de dados representando informações diversas, como o número de computadores, o número de cachorros, um peso, além de um nome e um sobrenome. Este vetor será uma coleção mista que inclui tanto números quanto strings.

```
# Criando um vetor em R
lista <- c(2, 3, 110, 7, 33, 'Joao', 'Guilherme')
lista
```

```
## [1] "2"          "3"          "110"        "7"          "33"         "Joao"
## [7] "Guilherme"
```

Descrição dos elementos: * 2 representa o número de computadores. * 3 é o número de cachorros. * 110 pode ser interpretado como um peso em quilogramas. * 7 indica o número de membros na casa. * 33 poderia ser a idade de uma pessoa. * 'Joao' é um nome. * 'Guilherme' é um sobrenome.

```
# Verificando o primeiro elemento da lista
if (lista[1] >= 1) {
  print(paste("Ele tem", lista[1], "computador(es)"))
} else {
  print("Ele não tem computadores")
}
```

```
## [1] "Ele tem 2 computador(es)"
```

```
# Verificando o segundo elemento da lista
if (lista[2] > 0) {
  print(paste("Ele tem", lista[2], "cachorros"))
} else {
  print("Ele não tem cachorro")
}
```

```
## [1] "Ele tem 3 cachorros"
```

```
# Verificando o terceiro elemento da lista para categorizar o peso
if (lista[3] >= 80 && lista[3] <= 100) {
  print("Sobrepeso")
} else if (lista[3] >= 60 && lista[3] <= 79) {
  print("Normal")
} else {
  print("Obesidade")
}
```

```
## [1] "Obesidade"
```

Estrutura de Repetição

As estruturas de repetição permite a execução repetida de um bloco de código, dependendo de condições específicas ou de um número definido de vezes. Em R, as principais estruturas de repetição são:

- **for:** O loop `for` em R é usado para iterar sobre elementos em uma sequência, como vetores, listas ou linhas de um data frame. É frequentemente utilizado quando sabemos o número de iterações ou quando estamos iterando sobre uma coleção de dados.
- **while:** O loop `while` executa um bloco de código enquanto uma condição específica é verdadeira. É útil quando o número de iterações não é conhecido antecipadamente.

```
# Loop for com seq() em R
for (i in seq(10)) {
  print(i)
}
```

For em R

```
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
## [1] 6
## [1] 7
## [1] 8
## [1] 9
## [1] 10
```

```
# Iterando diretamente sobre os elementos de um vetor
lista <- c(2, 3, 110, 7, 33, 'Joao', 'Guilherme')
for (item in lista) {
  print(item)
}
```

```
## [1] "2"
## [1] "3"
## [1] "110"
## [1] "7"
## [1] "33"
## [1] "Joao"
## [1] "Guilherme"
```

```
# Iterando sobre os índices de um vetor
for (i in 1:length(lista)) {
  print(lista[i])
}
```

```
## [1] "2"
## [1] "3"
## [1] "110"
## [1] "7"
## [1] "33"
## [1] "Joao"
## [1] "Guilherme"
```

```
# Loop while em R
validade <- 35
while (validade > 0) {
```

```

print(paste("Seu produto vence em", validade, "dias."))
validade <- validade - 1
}

```

While em R

```

## [1] "Seu produto vence em 35 dias."
## [1] "Seu produto vence em 34 dias."
## [1] "Seu produto vence em 33 dias."
## [1] "Seu produto vence em 32 dias."
## [1] "Seu produto vence em 31 dias."
## [1] "Seu produto vence em 30 dias."
## [1] "Seu produto vence em 29 dias."
## [1] "Seu produto vence em 28 dias."
## [1] "Seu produto vence em 27 dias."
## [1] "Seu produto vence em 26 dias."
## [1] "Seu produto vence em 25 dias."
## [1] "Seu produto vence em 24 dias."
## [1] "Seu produto vence em 23 dias."
## [1] "Seu produto vence em 22 dias."
## [1] "Seu produto vence em 21 dias."
## [1] "Seu produto vence em 20 dias."
## [1] "Seu produto vence em 19 dias."
## [1] "Seu produto vence em 18 dias."
## [1] "Seu produto vence em 17 dias."
## [1] "Seu produto vence em 16 dias."
## [1] "Seu produto vence em 15 dias."
## [1] "Seu produto vence em 14 dias."
## [1] "Seu produto vence em 13 dias."
## [1] "Seu produto vence em 12 dias."
## [1] "Seu produto vence em 11 dias."
## [1] "Seu produto vence em 10 dias."
## [1] "Seu produto vence em 9 dias."
## [1] "Seu produto vence em 8 dias."
## [1] "Seu produto vence em 7 dias."
## [1] "Seu produto vence em 6 dias."
## [1] "Seu produto vence em 5 dias."
## [1] "Seu produto vence em 4 dias."
## [1] "Seu produto vence em 3 dias."
## [1] "Seu produto vence em 2 dias."
## [1] "Seu produto vence em 1 dias."

```

```

# Iterando sobre os elementos de um vetor com while
i <- 1
while (i <= length(lista)) {
  print(paste("Item da lista", i, ":", lista[i]))
  i <- i + 1
}

```

```

## [1] "Item da lista 1 : 2"
## [1] "Item da lista 2 : 3"
## [1] "Item da lista 3 : 110"
## [1] "Item da lista 4 : 7"

```

```
## [1] "Item da lista 5 : 33"  
## [1] "Item da lista 6 : Joao"  
## [1] "Item da lista 7 : Guilherme"
```